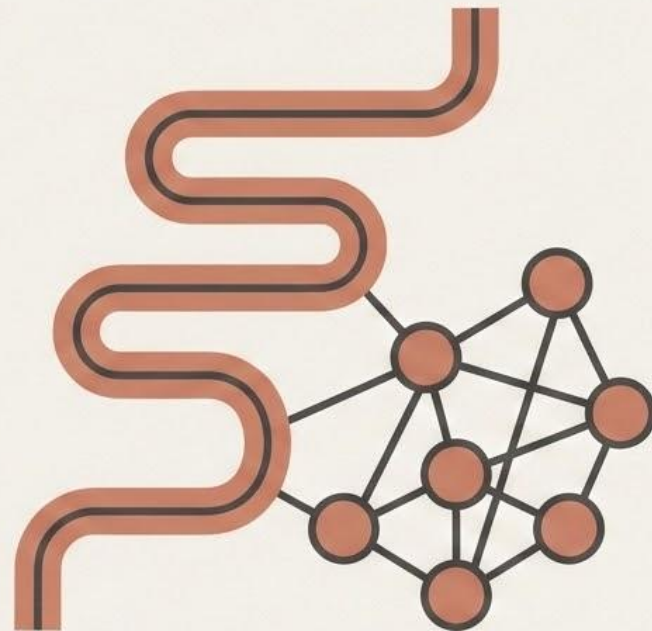


HANDLING LONG-TERM DEPENDENCIES WITH GATED RNNs AND LEARNING ON GRAPHS

DURATION: 1 Hour

TARGET AUDIENCE: Data Science/Computer Science students

PREREQUISITES: Understanding of basic RNNs, backpropagation, deep learning fundamentals



The Memory Problem in Deep Learning

Moving Beyond Basic RNNs



The Challenge:

- Basic Recurrent Neural Networks (RNNs) struggle to retain information from many steps ago.



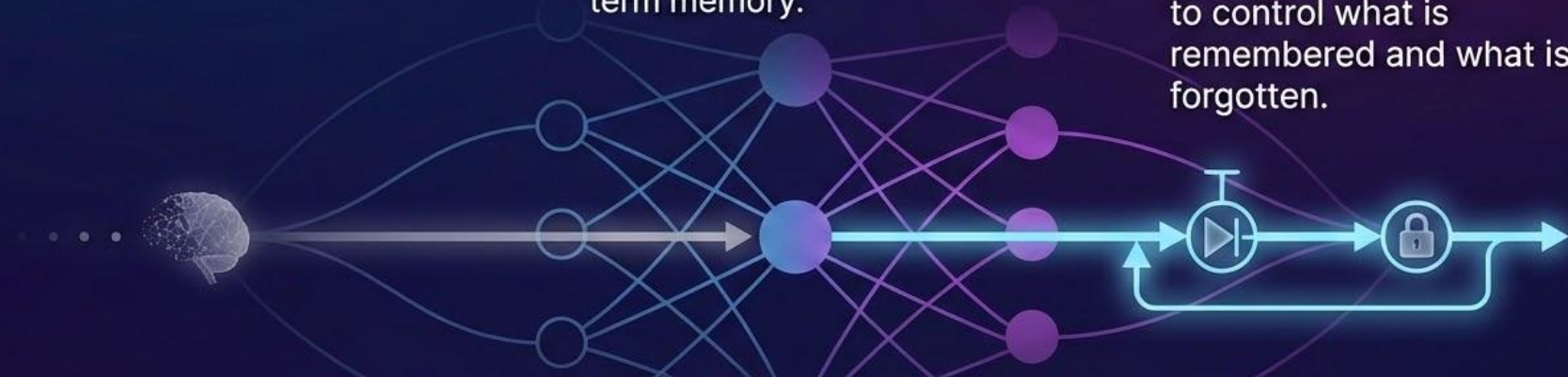
The Reality:

- Real-world data like human language, continuous video streams, and financial time series demand robust, long-term memory.



The Solution:

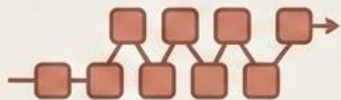
- Introduction of Gated Architectures (LSTM and GRU) equipped with sophisticated mechanisms to control what is remembered and what is forgotten.



The Evolving Data Landscape

Introduction to Graph-Structured Data

Traditional Data Formats



1D Sequences
(Text, Audio)



2D/3D Grids
(Images, Video)

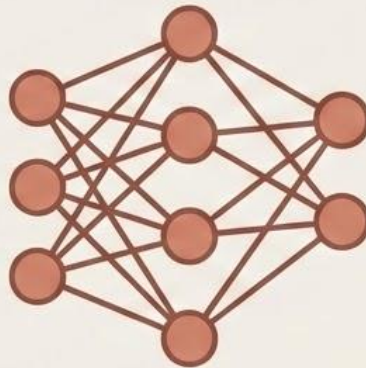
The **Unstructured** Reality

What happens when data doesn't fit neatly into a sequence or a grid?

- Molecular structures in drug discovery.
- Complex social networks and recommendation systems.
- Logistics and traffic networks.



The Next Frontier: **Graph Neural Networks (GNNs)** extend deep learning capabilities natively into graph-structured data.



Roadmap for the Session

Learning Objectives: What We Will Master Today

By the end of this lecture, you will be able to:

- Identify the mathematical and structural limitations of basic RNNs, specifically the need for gated architectures.
- Master the mechanics of LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) networks for handling long-term dependencies.
- Apply advanced techniques for addressing the vanishing gradient problem beyond just using gated RNNs.
- Understand the foundational concepts of Graph Neural Networks (GNNs) and how they process graph-structured data.
- Explore specific GNN variants, including Graph Convolutional Networks (GCNs) and their real-world applications.

THE MEMORY BOTTLENECK

The Problem with Basic RNNs: Hitting the 10-Step Wall



The Core Issue: Vanishing Gradients

As errors propagate backward through time, the gradients shrink exponentially, preventing the network from learning long-range dependencies.



The Practical Limit: ~10 Time Steps

In practice, standard RNNs can only "remember" information from roughly ~10 time steps back.



The Result: Failure to Capture Long Sequences

They fail to capture complex relationships across long sequences (e.g., tying the end of a long paragraph back to its opening sentence).



The Solution: Gated RNNs

These introduce sophisticated internal mechanisms to explicitly control information flow—allowing the network to actively decide what to remember, what to forget, and what to output.

Long Short-Term Memory (LSTM)

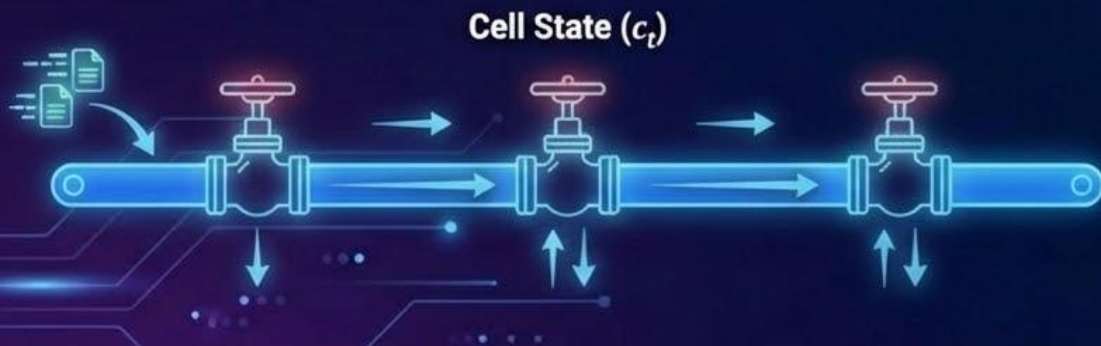
The Conveyor Belt of Memory



Origins: Invented by Sepp Hochreiter and Jürgen Schmidhuber in 1997.

Impact: The foundational architecture that enabled practical, modern deep learning for sequential data.

The Core Innovation: The Cell State (c_t)



Acts as a structural “conveyor belt” running straight down the entire sequence chain.



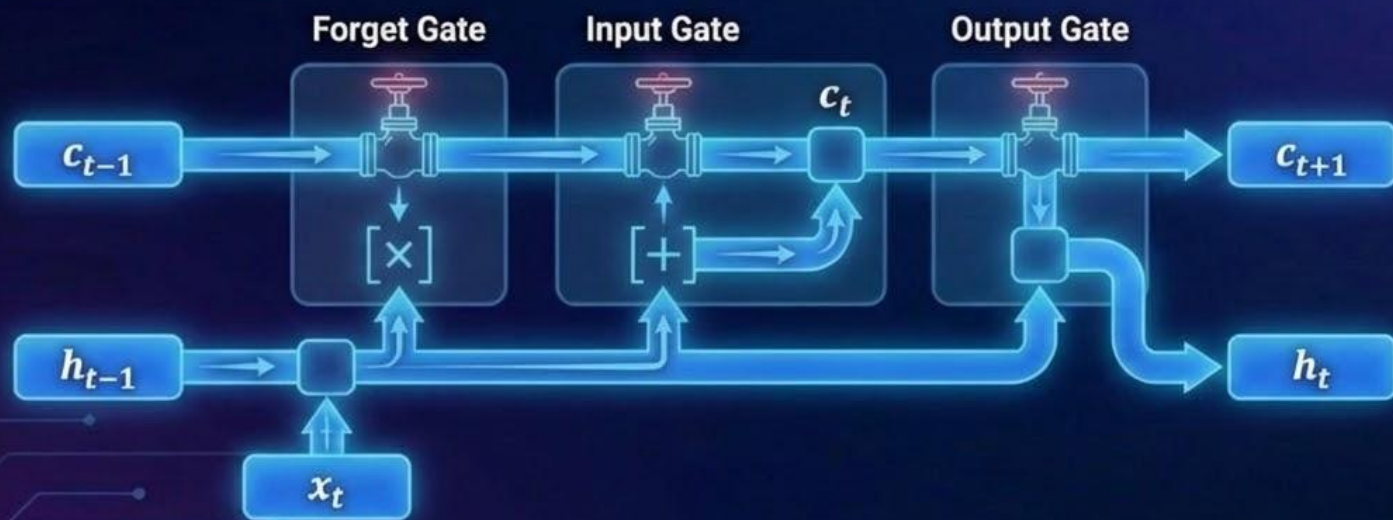
Information can flow along it completely unchanged, preventing gradients from vanishing.



Access to this conveyor belt is strictly controlled by three distinct gates.

Inside the Mechanism

LSTM Architecture: Visualizing the Three Gates



Forget Gate:

Decides what information to discard from the past cell state.



Input Gate:

Decides what new information to store in the cell state.



Output Gate:

Decides what part of the cell state should be output as the new hidden state (h_t).

The Gate Equations

Filtering Information

The gates use sigmoid (σ) activations to output values between 0 (block completely) and 1 (pass completely).

1. The Forget Gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

(What old information should we drop?)

2. The Input Gate:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

(Which specific values should we update?)

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

(What is the candidate new information we want to add?)

3. The Output Gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

(What information should we reveal to the next layer?)



Updating the State

The Final Calculation

Creating the New Memory

Cell State Update (The Conveyor Belt):

$$c_t = \underbrace{f_t * c_{t-1} + i_t * \tilde{c}_t}_{\text{Translation: (Forget vector * Old Memory) + (Input filter * New Candidate Data)}} \rightarrow \text{Conveyor Belt with } c_t$$

Translation: (Forget vector * Old Memory)
+ (Input filter * New Candidate Data)

Hidden State Update (The Output):

$$h_t = \underbrace{o_t * \tanh(c_t)}_{\text{(Output filter * Tanh(New Cell State))}}$$

(Output filter * Tanh(New Cell State))

The Core Intuition:

- $\tilde{c}_t$ c_t carries the long-term context across many steps.
-  The gates continuously learn to preserve the relevant signal and discard the noise.
-  Because the main update to c_t involves simple addition rather than complex matrix multiplication, gradients flow backward easily, solving the vanishing gradient problem.



The Gated Recurrent Unit (GRU)

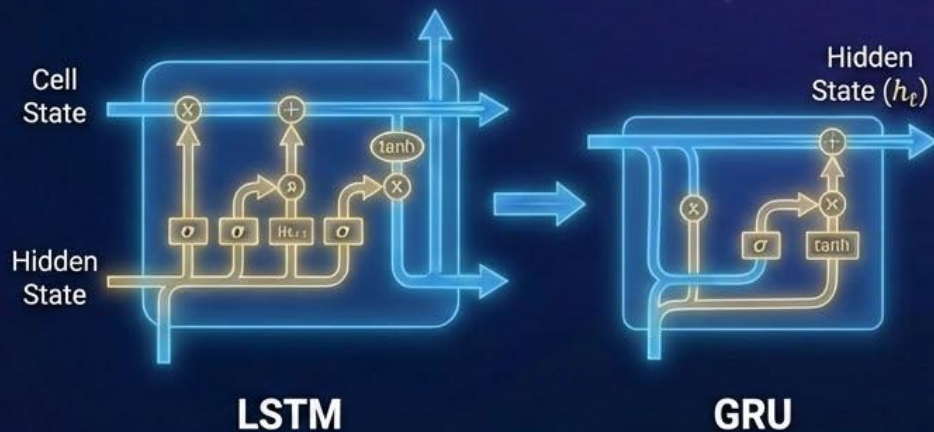
Efficiency Without Sacrificing Memory

Origins: Introduced by Kyunghyun Cho and his team in 2014.

The Goal: Simplify the complex LSTM architecture while maintaining its ability to capture long-term dependencies.

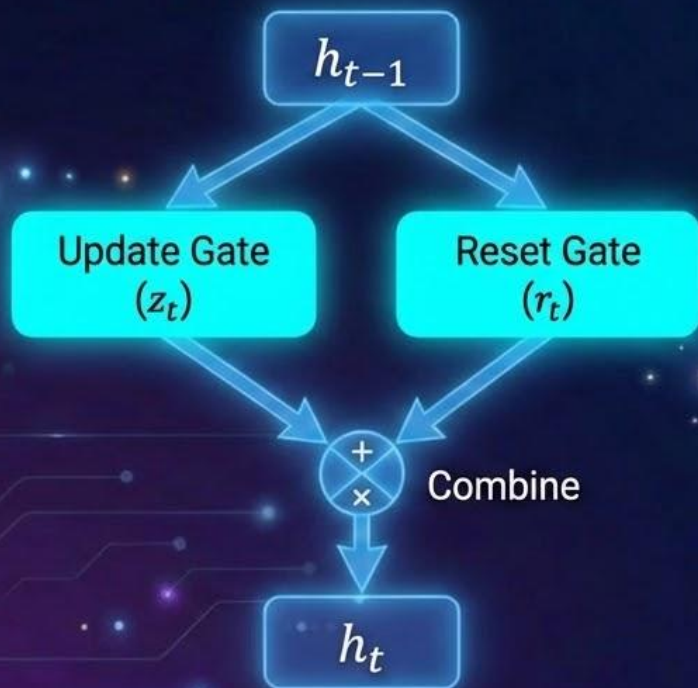
The Core Difference:

- LSTMs use a separate Cell State and Hidden State.
- GRUs merge them into a single Hidden State (h_t).
- GRUs reduce the number of gates from three to two.



GRU Architecture

The Two-Gate System



1. The Update Gate (z_t):

- Replaces both the Forget and Input gates of the LSTM.
- It decides how much of the past information to keep versus how much new information to add.

- 2. The **Reset Gate (r_t)**: Decides how
 - much of the past information to completely ignore when creating the new candidate memory.

The GRU Equations

Calculating the Hidden State

1. The Gates (Sigmoid Filters):

- Update Gate:
- Reset Gate:

2. The Candidate State:

- What new information do we want to add? (Notice the Reset Gate r_t actively masking the past state h_{t-1}).

3. The Final Hidden State:

- The linear interpolation between old memory and new data.

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W_h \cdot [r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Choosing Your Architecture

LSTM vs. GRU – Which Should You Use?

Aspect	LSTM	GRU
 Parameters / Math	More (Requires 4 gate calculations)	Fewer (Requires 2 gate calculations)
 Memory Structure	Separate cell state (c_t) & hidden state	Hidden state only (h_t)
 Performance	Slightly better on highly complex tasks	Comparable, but with faster training times
 When to Use	Large datasets, extreme long-range dependencies	Smaller datasets, or when compute efficiency matters

Why Gated RNNs Work

Solving the Mathematical Bottleneck

Gradient Flow



By relying on linear addition (like the $(1 - z_t)$ interpolation in GRUs or the $+ i_t$ in LSTMs) rather than just matrix multiplication, gradients can flow backwards across hundreds of steps without vanishing.

Learnable Context



The network doesn't rely on on a hardcoded memory window. The gates learn dynamically exactly which information to preserve based on the data.

Empirical Dominance



These architectures were the undisputed **state-of-the-art** for almost all sequential tasks (translation, speech, text generation) until the invention of the **Transformer**.

Stabilizing the Gradient

Preventing Explosions and Vanishing Acts

Even with gated architectures, training deep RNNs requires mathematical guardrails.



1. Gradient Clipping

The Problem: Gradients can compound and become massive, causing the network weights to swing wildly out of control (Exploding Gradients).

The Fix: Cap the gradients at a strict maximum value. (e.g., If the math calculates a gradient of 85, the system simply overwrites it to 5.0).

Result: Essential for stable RNN training.



2. Orthogonal Initialization

The Fix: Initialize the starting recurrent weights as orthogonal matrices rather than random numbers.

Result: Geometrically preserves the magnitude of the gradient, keeping the weight multiplier close to 1.0, which inherently fights both vanishing and exploding gradients.

Changing the Flow of Time

Sequence Processing Strategies: Optimizing How We Read the Data



3. Truncated Backpropagation Through Time (TBPTT)

The Problem: Backpropagating an error through a 10,000-word document takes too much memory.

The Fix: Process the sequence in manageable chunks. Backpropagate only through a limited window (e.g., 100 steps at a time).

Trade-off: Saves memory, but sacrifices the ability to learn dependencies longer than the window size.



4. Bidirectional RNNs

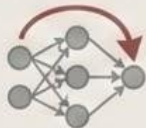
The Fix: Process the sequence both forward and backward simultaneously.

Mechanism: Maintains two separate hidden states: $\vec{h}_t$ (forward) and $\overleftarrow{h}_t$ (backward). These are concatenated for the final representation.

Limitation: Only works when the full context is available (e.g., translating a completed document, not real-time speech).

Structural Enhancements

Easing the Burden on the Network



5. Skip Connections

Mechanism: Create direct neural connections that jump across multiple time steps.

Result: Allows gradients to completely bypass the long sequential chain, providing a “fast lane” for error signals to reach early layers without vanishing.



6. Layer Normalization

Mechanism: Normalizes the neural activations across the features within each individual time step.

Result: Stabilizes the training process by ensuring the data flowing through the network maintains a consistent scale and variance, which is particularly vital for very deep RNNs.

The Precursor to the Future Attention Mechanisms

The Bridge to Modern Deep Learning



7. Attention (Pre-Transformer Era):

The Problem: A standard RNN tries to compress an entire sentence into a **single final hidden state vector** (the “**compression problem**”).

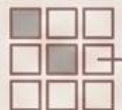
The Fix: Give the network a mechanism to look back and **directly access all previous hidden states**.

Mechanism: Computes a **weighted sum of all past hidden states** based on their **relevance to the current step**.

The Impact: It solved the compression bottleneck and paved the exact path for the invention of the **Transformer architecture**.

The Limitation of CNNs and RNNs

Adapting deep learning for unstructured data.



CNN Assumption:

Grid structures (images, video).



RNN Assumption:

Sequential structures (text, time series).



The Problem:

Many real-world datasets are graphs and lack a natural, fixed ordering.



The Need:

A neural network architecture designed for graph-structured data.

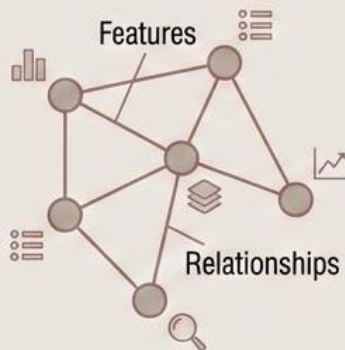
What is a Graph?

Nodes and Edges

The Definition:

$$G = (V, E)$$

- **V**: Nodes (vertices) with features.
- **E**: Edges representing relationships.



Examples of Graph Data:

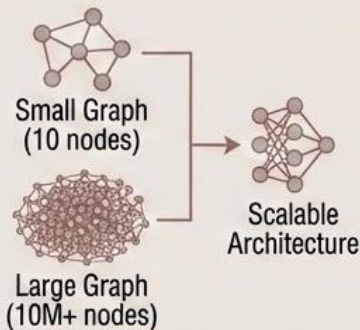
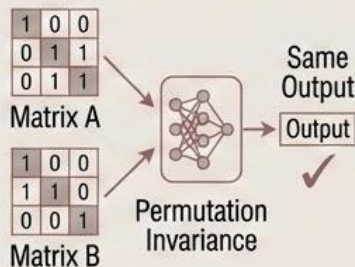
Domain	Nodes	Edges	Node Features
Social Network	Users	Friendships	Profile data, interests
Molecules	Atoms	Chemical bonds	Atom type, atomic charge
Citation Network	Papers	Citations	Text content, abstract
Knowledge Graph	Entities	Relationships	Entity type, properties
Traffic Network	Intersections	Roads	Traffic sensor data

The Graph Learning Challenge

Permutation Invariance

Why Traditional NN Architectures Fail

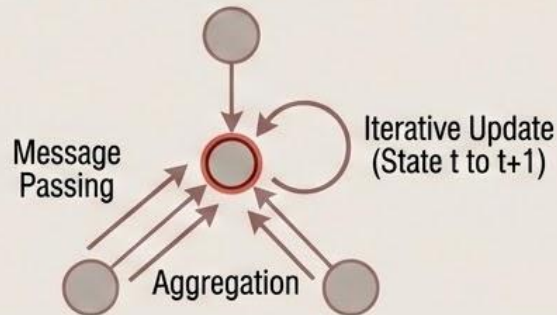
- **No Fixed Node Ordering:**
A graph can be represented by many different adjacency matrices; the network's output must be the same regardless of which node is "first."
(Permutation Invariance)
- **Variable Graph Size:**
Networks must handle graphs with 10 nodes or 10 million nodes without breaking.



Graph Neural Network Core Idea

- **Aggregation and Iterative Updates:** GNNs use local aggregation, where nodes collect and summarize information from their immediate neighbors (message passing), updating their own representations iteratively.

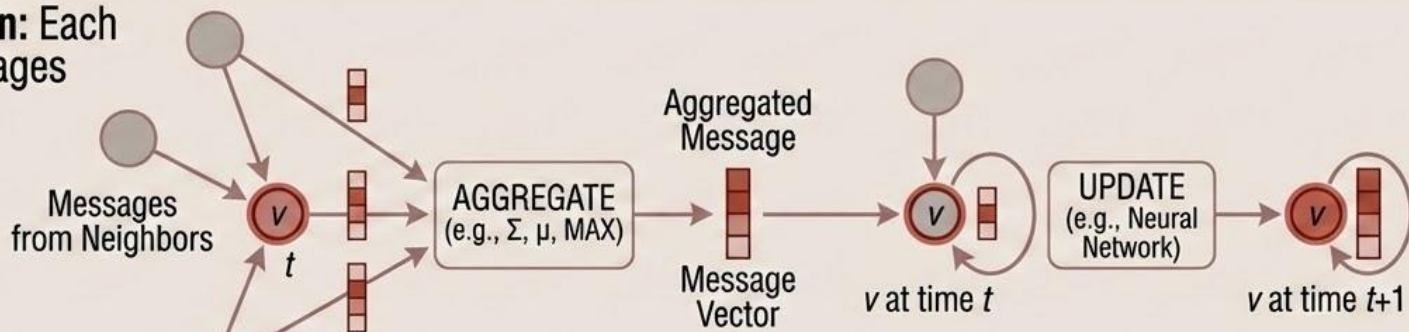
Local Aggregation & Message Passing



The Message Passing Framework

GNNs iteratively update node representations using a structured process called message passing. This is how information propagates throughout the graph.

- 1. **Message Collection:** Each node v gathers messages from all of its direct neighbors.



- 2. **Message Aggregation:** These multiple messages are summarized into a single "aggregated message" vector using a function like SUM, MEAN, MAX, or a more complex attention mechanism.

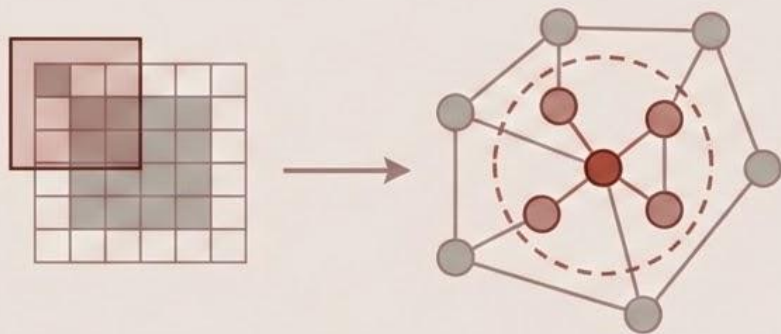
- 3. **Representation Update:** The node v combines its original state with this aggregated message to compute its new, updated representation for the next iteration.

Graph Convolutional Networks (GCNs)

Aggregating Local Neighborhoods

The Core Concept:

- How do we apply a CNN-style “filter” when there is no grid? By treating a node’s immediate neighbors as its “receptive field.”

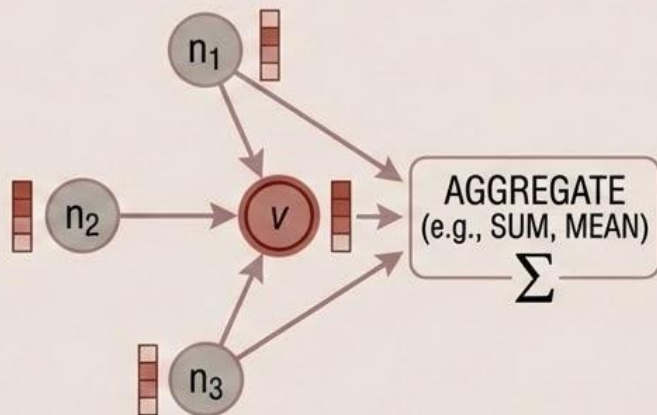


Grid Receptive Field

Graph Receptive Field
(Neighbors)

The GCN Approach:

- We extend the idea of convolution to graphs by having each node aggregate the feature vectors from its directly connected neighbors.



The Mathematics of GCNs

The GCN Layer Equation: Matrix Operations for Message Passing


$$\mathbf{H}^{(l+1)} = \sigma(\hat{\mathbf{D}}^{-1/2} \hat{\mathbf{A}} \hat{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)})$$

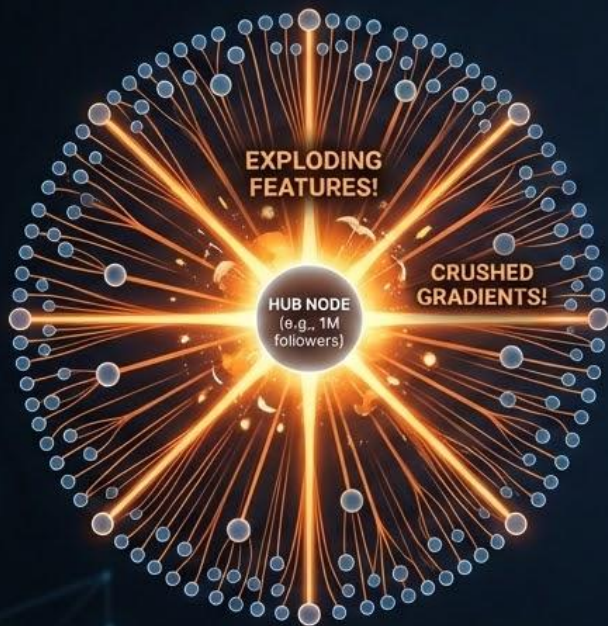
Decoding the Formula:

- $\mathbf{H}^{(l)}$: The matrix of all node features at the current layer (l).
- $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$: The Adjacency matrix (who is connected to who) plus self-loops (identity matrix $\mathbf{I}$), ensuring a node includes its own previous features in the update.
- $\hat{\mathbf{D}}$: The Degree matrix (how many connections each node has).
- $\mathbf{W}^{(l)}$: The learnable weight matrix (the actual neural network parameters).
- σ : The non-linear activation function (e.g., ReLU).

Why Normalize? The Normalization Sandwich

Preventing Exploding Features in Graph Convolutional Networks

The Problem with Simple Summation

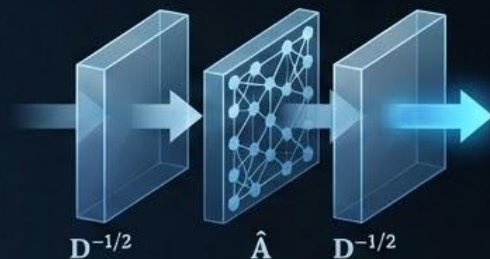


- Massive 'hub' nodes cause summed feature values to explode, leading to numerical instability.

$$\dots \mathbf{D}^{-\frac{1}{2}} \hat{\mathbf{A}} \mathbf{D}^{-\frac{1}{2}} \dots$$

The Core Normalization Term

The Solution: Symmetric Normalization



- Sandwiches the Adjacency matrix ($\hat{\mathbf{A}}$) between inverse square roots of the Degree matrix ($\mathbf{D}$).

Intuition: Scaling Messages by Connectivity



- Scales down messages from high-degree nodes. A message from a node with 1 connection is 'louder' than one from 1,000.

The Micro Perspective

A Node's Point of View

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\mathbf{W}^{(l)} \cdot \text{MEAN}_{u \in \mathcal{N}(v) \cup \{v\}} \mathbf{h}_u^{(l)} \right)$$



(A node's new state is the weighted average of its neighbors and itself).

Core GCN Properties:



- **Permutation Invariant:** The mathematical output is identical regardless of the order in which you process the neighbors.



- **Scalable:** Because it relies on sparse matrix multiplication, it scales efficiently to massive graphs.



- **Dynamic Multi-Hop:** Stacking 3 GCN layers allows a node to gather information from neighbors 3 hops away.

Applications of GCNs

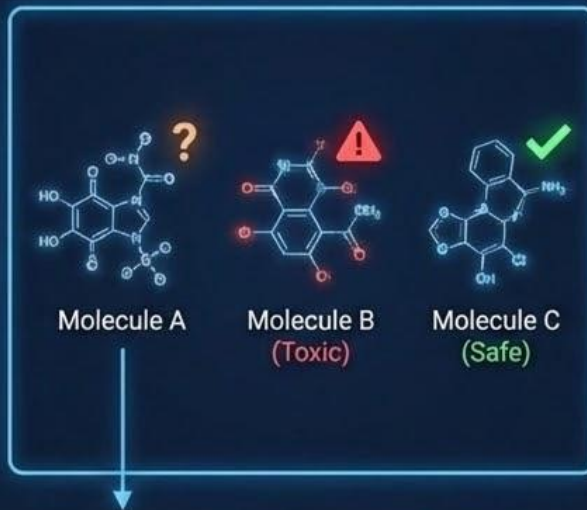
What Can We Predict?

Node Classification



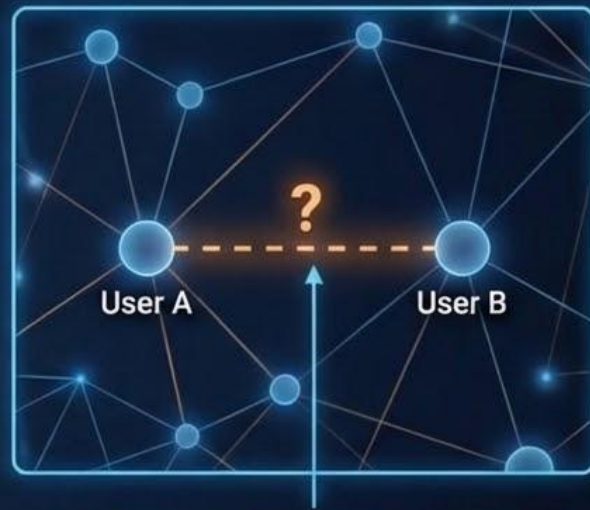
- Predicting the label of specific **nodes within a larger graph**.
- Example: Identifying fraudulent accounts within a massive financial transaction network.

Graph Classification



- Predicting a single label for an **entire, distinct graph**.
- Example: Determining if a specific synthesized molecular structure (represented as a graph) is toxic or safe.

Link Prediction

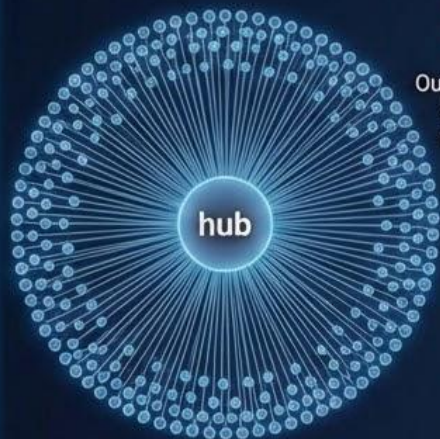


- Predicting whether an edge should exist between two nodes.
- Example: Suggesting "People You May Know" on a social network, or recommending a movie to a user.

GraphSAGE

Sample and AggregAGE

The Problem with Standard GCNs

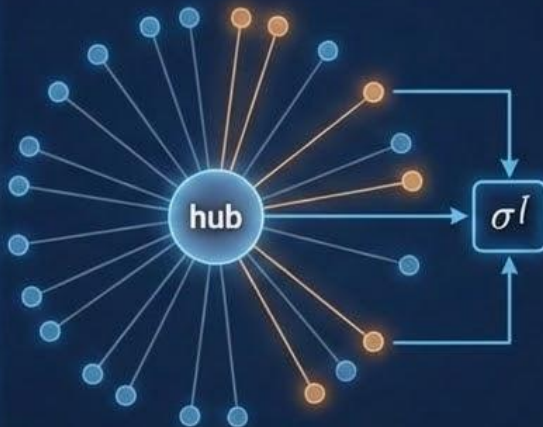


CRASH

Out of Memory
for 10M+
neighbors.

- To update a node, a standard GCN must load all its neighbors into memory. Hub nodes with millions of connections cause system crashes.

The GraphSAGE Solution



- * Instead of using the full neighborhood, it samples a fixed-size number of neighbors.

Result & Flexibility



 Pinterest
 twitter

- Makes deep learning on massive, billion-node graphs (like Pinterest or Twitter) computationally feasible.



Mean



Pooling



LSTM

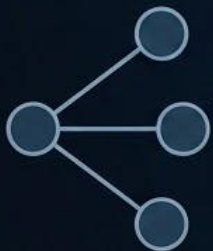
- Allows for multiple different aggregation functions, including Mean, Pooling, and even passing neighbors through an LSTM.

Graph Attention Networks (GAT)

Learning Neighbor Importance



THE CORE CONCEPT



Standard GCN:
All neighbors
treated equally

In a standard GCN, all neighbors are treated relatively equally (normalized by degree). In reality, not all connections carry the same importance.



THE GAT MECHANISM



Uses **Attention Mechanisms** to dynamically assign “weights” to **different neighbors**. These attention coefficients are not hardcoded; they are learned by the neural network during training.



THE IMPACT



A node can learn to heavily weigh the message from a **close** collaborator while completely ignoring the message from a distant acquaintance.

Advanced Graph Architectures

GGNN and MPNN

Gated Graph Neural Networks (GGNN)



- Introduces the time-series memory of RNNs into graphs.
- Uses GRU-like gating mechanisms to update node states iteratively for a fixed number of steps.
- Highly effective for complex graph-level tasks and sequential processes on graphs.

Message Passing Neural Networks (MPNN)



- Not a single network, but a general theoretical framework.
- Unifies GCN, GraphSAGE, GAT, and others under three formal steps: Message function, Update function, and Readout function.

Expressiveness and Edge Features

GIN and ECC

Graph Isomorphism Network (GIN)



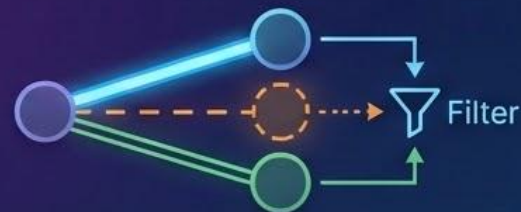
Maximum
Expressiveness

- **The Goal:** Maximum theoretical expressiveness.
- **The Benchmark:** Mathematically proven to be exactly as powerful as the Weisfeiler-Lehman test (the gold standard algorithm for determining if two graphs have identical structures).



WL Test
Gold
Standard

Edge-Conditioned Convolution (ECC)



- **The Goal:** Utilizing complex edge data. Standard GNNs focus mostly on Node features. ECC ensures that the convolutional filters adapt based on Edge attributes.
- **Example:** Crucial in chemistry, where the type of chemical bond (single, double, aromatic edge) drastically changes the interaction between atoms (nodes).



GNN Architecture Comparison

Aligning Models to Workloads

GNN Type	Key Feature / Mechanism	Best Used For	Scalability
GCN	 Normalized neighbor aggregation	Node classification	Moderate
GraphSAGE	 Fixed-size neighborhood sampling	Billion-node massive graphs	Excellent
GAT	 Attention weights on edges	Graphs with varying relationship importance	Moderate
GIN	 Maximum structural expressiveness	Whole-Graph classification	Moderate
GGNN	 Gated (GRU) memory updates	Sequential graph processes	Moderate

Key Formulas to Remember

The Equations that Drive the Networks

1. LSTM Cell Update (The Conveyor Belt)

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

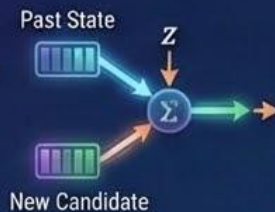
(New memory is the filtered old memory plus the filtered new candidate data).



2. GRU Hidden State Update

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

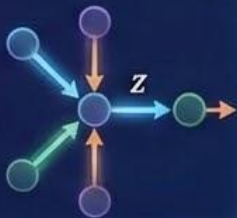
(A linear interpolation blending the past state and the new candidate).



2. GRU Hidden State Update

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

(A linear inters normalized aggregation of neighboring node features).



3. GCN Layer (Message Passing)

$$H^{(l+1)} = \sigma(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)})$$

(Symmetrically normalized aggregation of neighboring node features).



Key Insights & Final Takeaways

Structuring Data for Deep Learning



Overcoming the Bottleneck: Gated RNNs (LSTM/GRU) solve the vanishing gradient problem via controlled information flow.



LSTM vs. GRU: LSTMs utilize a completely separate memory cell, whereas GRUs are architecturally simpler but offer highly comparable performance.



The Graph Frontier: Graph Neural Networks extend the power of deep learning to non-grid, non-sequence, unstructured data.



The Unifying Principle: Message passing is the heart of all GNNs—nodes continuously learn by aggregating information from their local neighbors.

Data Dictates Architecture:



- Sequences / Time-Series → RNNs / LSTMs / GRUs
- Interconnected Relationships → GNNs / GCNs